

CZ2006/CE2006 Software Engineering

Lecture 18: Boundary Value Testing

Liu Yang

Equivalence Class Testing – Partition Input Values into Equivalence Classes

- ▶ A set of input values forms an **equivalence class (EC)** if
 - ▶ They all are supposed to produce the **same output**
 - ▶ If one value catches a bug, the others probably will too
 - ▶ If one value does not catch a bug, the others probably will not either



Equivalence Class Testing Process

► The tester

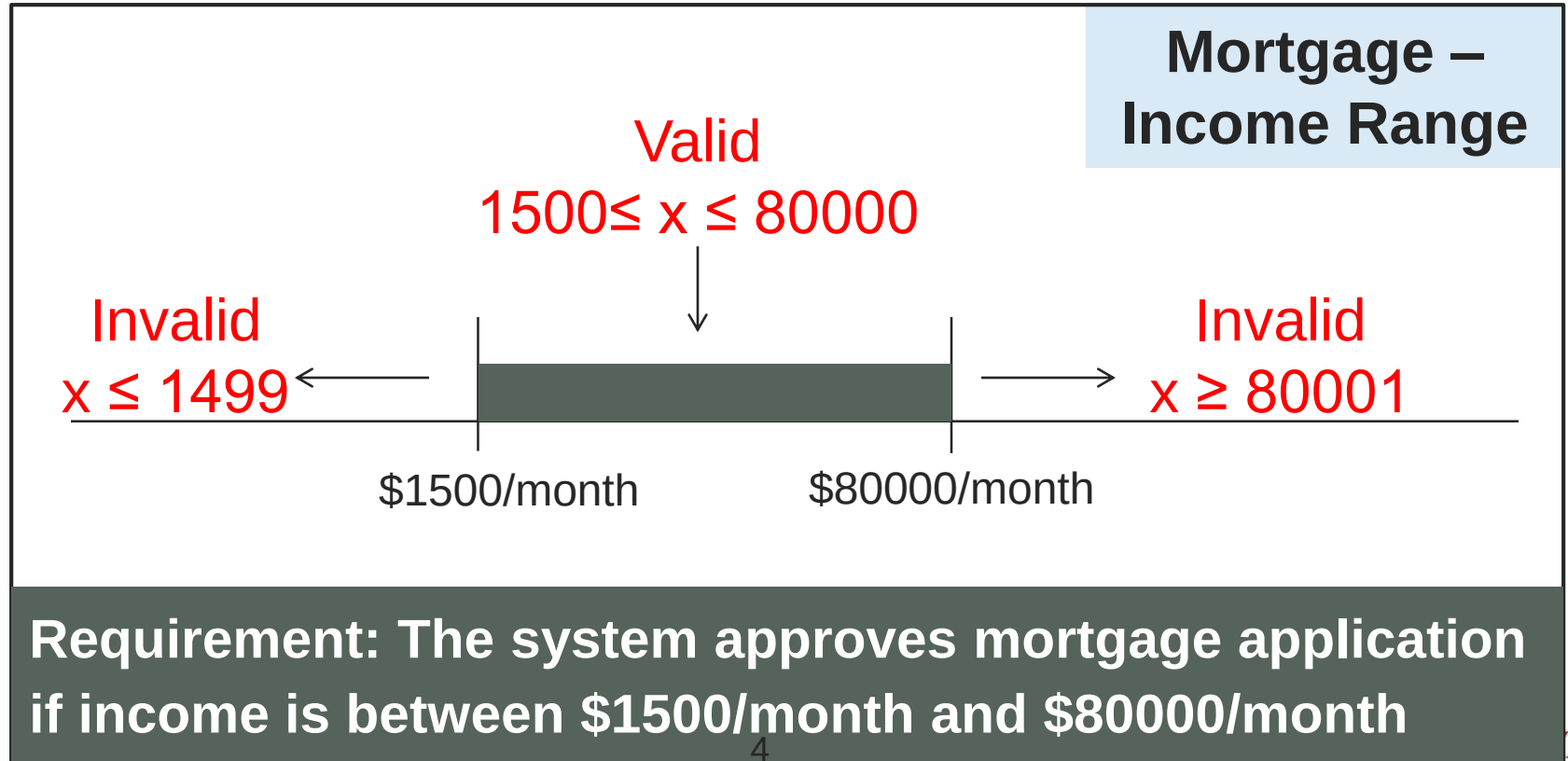
1. Identify the **valid and invalid equivalence classes** that partition the input values
2. Choose **at least one input value** for each equivalence class of each input parameter
3. Determines expected outputs (oracle) for chosen inputs
4. Constructs tests with the chosen inputs (e.g., write JUnit program)

► The testing engine (e.g., JUnit)

1. Executes the tests
2. Compares actual outputs (log) with the expected outputs (oracle)
3. Determines whether the SUT (System Under Test) functions properly

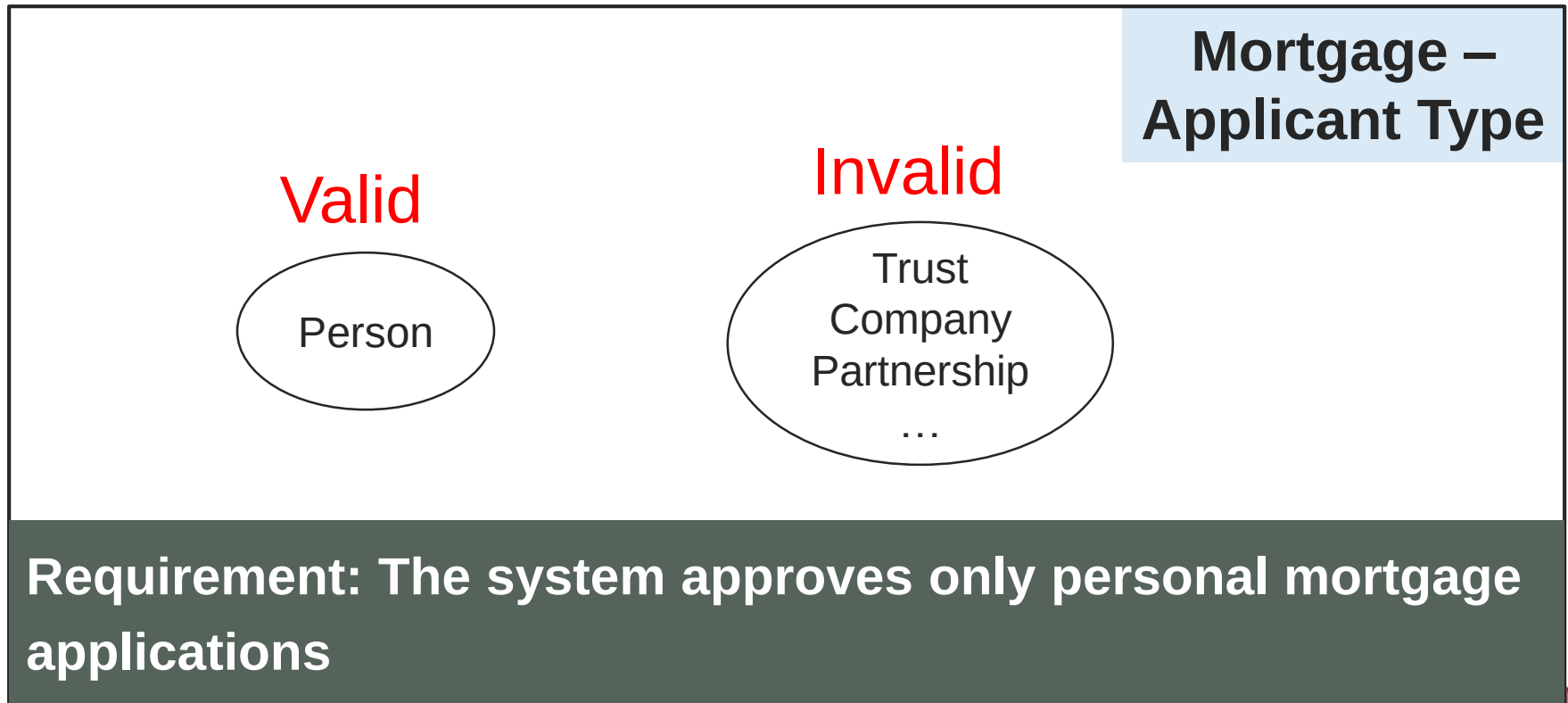
Range of Values

- ▶ **ONE** equivalence class of **VALID** values
- ▶ **TWO** equivalence classes of **INVALID** values



Discrete Values

- ▶ **ONE** equivalence class of **VALID** values
- ▶ **ONE** equivalence class of **INVALID** values



Design Test Cases Using ECs

1. Test **VALID** inputs of several parameters
 - ▶ Test valid inputs of several parameters at the same time (i.e. per test case)
 - ▶ Choose one valid input for each input parameter from an equivalence class
2. Test **INVALID** inputs of several parameters
 - ▶ Test **ONLY ONE INVALID** input from an equivalence class of one input parameter at the same time (i.e. per test case)
 - ▶ Choose valid inputs from an equivalence class of all other input parameters

Test Valid Inputs Using ECs

- ▶ You may test **valid** input values of **several** input parameters at the same time
- ▶ You may create test cases for **all the values** in an valid equivalence class if the set of valid values in the equivalence class is **small**

e.g. The number of mortgages (0, 1, 2, 3, 4, 5)
Property type (HDB, Condo)

but not for income range, or characters of password

Test Valid Inputs – Example

Each row is a test case.
For each test case, choose one **valid** input value from a valid equivalence class of each parameter

As each parameter has only one valid EC, this single test case satisfies **EC testing principle**

Income	Number of mortgage	Applicant Type	Property Type	Expected Result
\$1500	0	Person	Condo	Approval
\$2000	1	Person	Condo	Approval
\$5000	2	Person	Condo	Approval
\$12345	3	Person	HDB	Approval
\$76543	4	Person	HDB	Approval
\$80000	5	Person	HDB	Approval

Try to cover a few more valid income value when enumerating #mortgage and PropertyType values

As #mortgage and PropertyType have **only 6 (0-5) and 2 (Condo, HDB)** valid values, we create several more test cases to cover **all** these values

Test Invalid Inputs Using ECs

Why?

- ▶ You must test **ONLY ONE INVALID** input value from an equivalence class of one input parameter per test case i.e. each test case should have no more than one invalid input

Income	Number of mortgage	Applicant Type	Property Type	Expected Result
\$100	8	Company	Mobilehome	Reject

Invalid input Invalid input Invalid input Invalid input

If we test only one invalid input for each test case, then we can learn the error handling or behavior for each type of invalid input

Test Invalid Inputs – Example

Each row is a test case.
Test **ONLY ONE INVALID**
input value **per test case**

Income	Number of mortgage	Applicant Type	Property Type	Expected Result
\$100	1	Person	Condo	Reject
\$81000	2	Person	HDB	Reject
\$5000	-1	Person	Condo	Reject
\$10000	8	Person	HDB	Reject
\$12345	3	Company	HDB	Reject
\$76543	4	Person	Mobilehome	Reject

Choose valid input values
for all other parameters

Boundary Value Testing (BVT) – Which Values to Test?

- ▶ BVT is **only applicable** to **range of values**
 - ▶ Focus on **boundaries** of the equivalence classes of range of values
 - ▶ Select a **minimal** set of **valid** and **invalid inputs** (values **on-the-boundary**, **just-below**, or **just-above** boundaries) to test
- ▶ What is the boundary values of parameter Property type?
 - ▶ Do **discrete values**, such as property type, have boundary values? **NO**

Boundary Value Testing (BVT) Process

► The tester

1. Identify the **lower and upper boundaries** of equivalence classes of the range-of-values parameters
2. Choose three input values for each boundary
 - One value **on-the-boundary**, one value **just-below**, and one value **just-above**
 - Just-below and just-above value depends on the value's unit
 - No need to duplicate the test cases if the just-below or just-above values fall into other ECs (including current EC)
3. Determines expected outputs (oracle) for chosen inputs
4. Constructs tests with the chosen inputs (e.g., write JUnit program)

Determine Boundary Values (BVs) – Example

- ▶ Income range valid equivalence class ($1500 \leq x \leq 80000$)
 - Lower boundary 1500: ~~1499~~, 1500, ~~1501~~
 - Upper boundary 80000: ~~79999~~, 80000, 80001
 - Valid boundary values: {1500, 80000}
 - Invalid boundary values: {1499, 80001}
- ▶ Number of mortgage valid equivalence class ($0 \leq x \leq 5$)
 - Lower boundary 0: ~~-1~~, 0, ~~1~~
 - Upper boundary 5: ~~4~~, 5, 6
 - Valid boundary values: {0, 5}
 - Invalid boundary values: {-1, 6}
- ▶ Length of password valid equivalence class ($4 \leq x \leq 8$)
 - Lower boundary 4: 3, 4, ~~5~~
 - Upper boundary 8: ~~7~~, 8, 9
 - Valid boundary values: {4, 8}
 - Invalid boundary values: {3, 9}

Determine Boundary Values (BVs) – Example

- ▶ Hiring rules ($0 \leq \text{age} \leq 15$) (Do not hire)
 - Lower boundary 0: ~~-1~~, 0, ~~1~~
 - Upper boundary 15: ~~14~~, 15, ~~16~~
- ▶ Hiring rules ($16 \leq \text{age} \leq 17$) (Part time)
 - Lower boundary 16: ~~15~~, 16, ~~17~~
 - Upper boundary 17: ~~16~~, 17, ~~18~~
- ▶ Hiring rules ($18 \leq \text{age} \leq 54$) (Full time)
 - Lower boundary 18: ~~17~~, 18, ~~19~~
 - Upper boundary 54: ~~53~~, 54, ~~55~~
- ▶ Hiring rules ($55 \leq \text{age} \leq 99$) (Do not hire)
 - Lower boundary 55: ~~54~~, 55, ~~56~~
 - Upper boundary 99: ~~98~~, 99, 100

Valid
boundary
values:
{0, 15,
16, 17,
18, 54,
55, 00}

Invalid
boundary
values:
{-1, 100}

Determine Boundary Values (BVs) – Example

- ▶ Add1Divide30000 ($-32768 \leq \text{input} \leq -30001$) (return -1)
 - Lower boundary -32768: ~~-32769~~, ~~-32768~~, ~~-32767~~
 - Upper boundary -30001: ~~-30002~~, ~~-30001~~, ~~-30000~~
- ▶ Add1Divide30000 ($-30000 \leq \text{input} \leq 29998$) (return 0)
 - Lower boundary -30000: ~~-30001~~, ~~-30000~~, ~~-29999~~
 - Upper boundary 29998: ~~29997~~, ~~29998~~, ~~29999~~
- ▶ Add1Divide30000 ($29999 \leq \text{input} \leq 32767$) (return 1)
 - Lower boundary 29999: ~~29998~~, ~~29999~~, ~~30000~~
 - Upper boundary 32766: ~~32766~~, ~~32767~~, ~~32768~~

Design Test Cases Using BVs

1. Test **VALID** inputs of several parameters
 - ▶ Test valid inputs of several parameters at the same time
 - ▶ Choose one valid input from an equivalence class of each input parameter
 2. Test **INVALID** inputs of several parameters
 - ▶ Test **ONLY ONE INVALID** input from an equivalence class of one input parameter **per test case**
 - ▶ Choose **VALID** inputs from an equivalence class of **all other** input parameters
- ▶ **Use the boundary values of the equivalence classes of range-of-values parameters**
- ▶ Value **on-the-boundary** as **VALID** input
 - ▶ **Just-below** or **just-above** values (if not in other ECs, including the current EC) as **INVALID** inputs

Black Box Testing – ECs and BVs

	Range of Values	Discrete Values
Equivalence Classes (ECs)	ONE Valid EC TWO Invalid ECs	ONE valid EC ONE Invalid EC

Range of values	Lower Boundary	Upper Boundary
Boundary Values (BVs) <i>(for each EC)</i>	ONE value on-the-boundary ONE value just-below ONE value just-above	ONE value on-the-boundary ONE value just-below ONE value just-above

Discrete Values	NO Boundary Values
Boundary Values (BVs) <i>(for each EC)</i>	

Black Box Testing – ECs and BVs

For test cases (use boundary values) –

- Test combination of **VALID** inputs (i.e. value **on-the-boundary**), one from each EC
- Test **ONE INVALID** input from an EC per test case, all others are VALID inputs => **One test case can contain AT MOST ONE invalid input**

- **Test combination of boundary values**

Example: (two parameters, A and B of range of values)

1. Valid A + Valid B → *combination of VALID inputs*
2. Valid A + Invalid B →
3. Invalid A + Valid B → } *combination of boundary values*

- Note: For VALID EC: **On-the-boundary**: valid
Just-below and **just-above**: invalid
- Remove **just-below** or **just-above** values from the boundary values of each EC (no need to duplicate the test cases) if the **just-below** or **just-above** values **fall into other ECs (including current EC)**